# REPORT

Reviewer report: PyCQA/isort#2788: Reuse the config of

an indented import section

Run isort-2788, tested 2026-11-14 12:00 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-14 12:57 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Reuse the config of an indented import section (PyCQA/isort#2788).

• What we observed: unverified behaviour claim: "The output does not change."; needs a probe (175 test(s) ran

on main, 175 on the PR; 0 new test(s) pass, 175 unchanged; probe model-probe2 has no measured line

coverage (no coverage data); it does not count as covering the change).

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 11.6 GB (of 15.6 GB).

• Setup check: free disk 92.6 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: isort python setup install instructions https://github.com/PyCQA/isort site:github.com/PyCQA/isort

• Query: isort python documentation install https://github.com/PyCQA/isort site:github.com/PyCQA/isort

• Query: isort python install requirements https://pypi.org/project/isort/ https://github.com/PyCQA/isort

site:pypi.org

• Source: isort - PyPI — https://pypi.org/project/isort/

• Source: GitHub - PyCQA/isort: A Python utility / library to sort imports. · GitHub — https://github.com/pycqa/isort

• Source: isort/docs/quick_start/0.-try.md at main · PyCQA/isort · GitHub —

https://github.com/PyCQA/isort/blob/main/docs/quick_start/0.-try.md

• Source: Create sharable configuration presets for isort · Issue #970 · PyCQA/isort · GitHub —

https://github.com/PyCQA/isort/issues/970

• Source: No CLI executable on Ubuntu 20.04 · Issue #1947 · PyCQA/isort · GitHub —

https://github.com/PyCQA/isort/issues/1947

• Source: isort/docs/configuration/github_action.md at main · PyCQA/isort · GitHub —

https://github.com/PyCQA/isort/blob/main/docs/configuration/github_action.md

• Setup plan: the same packages on main and pr.

• Setup plan: mypy-extensions is noted, installed with the project.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.11.0, >=3.10.0

• Installed hypothesis 6.170.15 from project (named by the error and listed in the project) on main and pr.

• Installed black 26.10.1 from project (named by the error and listed in the project) on main and pr.

• Before rerun: verified: 140 test(s) ran on main, 140 on the PR; 0 new test(s) pass, 139 unchanged; 1 test

failure(s) pre-existing and identical on both sides; 1 test module(s) fail to load on both sides

(tests.unit.test_main), so their tests did not run; probe: claim(s) C10, C11, C4, C9 proven

• After rerun: could_not_verify: unverified behaviour claim: "The output does not change."; needs a probe (175

test(s) ran on main, 175 on the PR; 0 new test(s) pass, 175 unchanged; probe model-probe2 has no measured

line coverage (no coverage data); it does not count as covering the change)

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (175 unchanged, 3 runs per side) | pytest-pr-1.log:43 «175 passed in 2.61s»

| P-C4 | The output does not change. (PR description) | proven | probe model-probe2, c4.output_hash: same on

both sides: "3727adff524e0616022eadd8f4af21a0778b29fc4c77bdfefd1afce2cbf5e4b7" |

probe-model-probe2-pr-1.jsonl:4 «"observation": "c4.output_hash"»

| P-C9 | I measured no slowdown on files with only top-level imports and on a new Config for each call. (PR

description) | proven | probe model-probe2, c9.output_hash: same on both sides:

"f3741fac2beb18228f36dd593f95a39c406e262bc98f19e9a503bcab095e5bda" | probe-model-probe2-pr-1.jsonl:5

«"observation": "c9.output_hash"»

| P-C10 | Import time did not change (40 rounds). (PR description) | proven | probe model-probe2, c10.all_same:

same on both sides: true | probe-model-probe2-pr-1.jsonl:1 «"observation": "c10.all_same"»

| P-C11 | Old and new outputs matched on 1,000,000 inputs. (PR description) | proven | probe model-probe2,

c11.combined_output_hash: same on both sides:

"0b91fe21d31a4799ca9b3ad54b969092d7a5166b894ccd2b9b069c5c3b0c50a3" |

probe-model-probe2-pr-1.jsonl:3 «"observation": "c11.combined_output_hash"»

Citation check (PASS): 5 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 175 / 175 | | Tests unchanged | 175 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | -

| Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe2 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe2.py, frozen

c563a749783b at 2026-11-14T06:34:46.761441+00:00): Test output of process function for correctness and

stability

• P-C4 (PR description): The output does not change. Expects the covered observations to stay the same.

• P-C9 (PR description): I measured no slowdown on files with only top-level imports and on a new Config for

each call. Expects the covered observations to stay the same.

• P-C10 (PR description): Import time did not change (40 rounds). Expects the covered observations to stay the

same.

• P-C11 (PR description): Old and new outputs matched on 1,000,000 inputs. Expects the covered observations

to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c10.all_same | P-C10 | as claimed | same on both sides: true | probe-model-probe2-main-1.jsonl:1

«"observation": "c10.all_same"»<br>probe-model-probe2-pr-1.jsonl:1 «"observation": "c10.all_same"»

| c10.first_output_hash | P-C10 | as claimed | same on both sides:

"3727adff524e0616022eadd8f4af21a0778b29fc4c77bdfefd1afce2cbf5e4b7" |

probe-model-probe2-main-1.jsonl:2 «"observation":

"c10.first_output_hash"»<br>probe-model-probe2-pr-1.jsonl:2 «"observation": "c10.first_output_hash"»

| c11.combined_output_hash | P-C11 | as claimed | same on both sides:

"0b91fe21d31a4799ca9b3ad54b969092d7a5166b894ccd2b9b069c5c3b0c50a3" |

probe-model-probe2-main-1.jsonl:3 «"observation":

"c11.combined_output_hash"»<br>probe-model-probe2-pr-1.jsonl:3 «"observation":

"c11.combined_output_hash"»

| c4.output_hash | P-C4 | as claimed | same on both sides:

"3727adff524e0616022eadd8f4af21a0778b29fc4c77bdfefd1afce2cbf5e4b7" |

probe-model-probe2-main-1.jsonl:4 «"observation": "c4.output_hash"»<br>probe-model-probe2-pr-1.jsonl:4

«"observation": "c4.output_hash"»

| c9.output_hash | P-C9 | as claimed | same on both sides:

"f3741fac2beb18228f36dd593f95a39c406e262bc98f19e9a503bcab095e5bda" |

probe-model-probe2-main-1.jsonl:5 «"observation": "c9.output_hash"»<br>probe-model-probe2-pr-1.jsonl:5

«"observation": "c9.output_hash"»

Citation check (PASS): 10 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (5.4 s run).

Coverage gate (python, confidence normal): 4 of 4 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| isort/core.py | 4 | 4

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested c9c319e6f946 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 4 of 4 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 5 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 10 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | unverified behaviour claim: "The output does not change."; needs a

probe (175 test(s) ran on main, 175 on the PR; 0 new test(s) pass, 175 unchanged; probe model-probe2 has no

measured line coverage (no coverage data); it does not count as covering the change) | Read the setup/build log

under Evidence links; build failures usually need a compiler or a newer nightly.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 4 of 4 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 5 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 10 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-14/isort-2788 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/PyCQA/isort/pull/2788

Evidence links

• Pull request PyCQA/isort#2788

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-probe2-main-1.jsonl

• probe-model-probe2-pr-1.jsonl

• main run 1 result: pytest-main-1.log:43 «175 passed in 3.07s»

• pr run 1 result: pytest-pr-1.log:43 «175 passed in 2.61s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-14 12:57 IST